

Nodejs session 7

▼ coding

```
//conrollers --> courses-conroler

const Course = require('../models/courses_module');
const asyncWrapper = require('../middlewere/asyncWrapper');

const {validationResult} = require('express-validator');

const httpStatusText = require('../utils/httpStatusText');

const AppError = require('../utils/appError');

const getAllCourse = async (req, res) => {
  const query = req.query;
  const limit = query.limit || 10;
  const page = query.page || 1;
  const skip = (page - 1) * limit;
  //get all courses from the database
  const courses = await Course.find({}, {"__v":false}).limit(limit).skip(skip);

  res.json({ status: httpStatusText.SUCCESS, data: { courses } });
}

const getCourseById = asyncWrapper(
  async (req, res, next) => {
    const course = await Course.findById(req.params.id, {"__v":false});
    if (!course){
      const error = AppError.create('Course not found',
```

```

    404, httpStatusText.FAILED);
    return next(error);
  }
  res.json({ status: httpStatusText.SUCCESS, data: { course } });
}
)

const postCreateCourse = asyncWrapper(
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      const error = AppError.create('Course not found', 404, httpStatusText.FAILED);
      return next(error);
    }
    const course = new Course(req.body);
    await course.save();
    res.status(201).json({ status: httpStatusText.SUCCESS, data: { course } });
  });

const updateCourseById = asyncWrapper(
  async (req, res, next) => {
    const course = await Course.findByIdAndUpdate(req.params.id, req.body, { new: true });
    if (!course) {
      const error = AppError.create('Course not found', 404, httpStatusText.FAILED);
      return next(error);
    }
    res.status(200).json({ status: httpStatusText.SUCCESS, data: { course } });
  }
);

```

```

);

const deleteCourseById = asyncWrapper(
  async (req, res, next) => {
    const course = await Course.findByIdAndDelete(req.
params.id);
    if (!course) {
      const error = AppError.create('Course not foun
d', 404, httpStatusText.FAILED);
      return next(error);
    }
    res.status(200).json({ status: httpStatusText.SUCC
ESS, data: { message: 'Course deleted' } });
  });

module.exports = {
  getAllCourse,
  getCourseById,
  postCreateCourse,
  updateCourseById,
  deleteCourseById
}

// middlewere --> asyncWrapper.js
module.exports = (asyncfn) => {
  return (req, res, next) => {
    Promise.resolve(asyncfn(req, res, next)).catch(nex
t);
  };
};

// models
const mongoose = require('mongoose');

const coursesSchema = new mongoose.Schema({
  title: {

```

```

        type: String,
        required: true
    },
    price: {
        type: Number,
        required: true
    }
})

module.exports = mongoose.model('Course', coursesSchema);

// rout --> courses_rout.js
const express = require('express');

const {body} = require('express-validator');

const router = express.Router();

const {
    getAllCourse,
    getCourseById,
    postCreateCourse,
    updateCourseById,
    deleteCourseById
} = require('../controllers/courses-controller');
// Get all courses
router.route('/')
    .get(getAllCourse)
    .post([
        body('title')
            .notEmpty()
            .isLength({ min: 2})
            .withMessage('Title must be at least 2 characters long'),
        body('price')
            .notEmpty()

```

```

        .withMessage('Price must be a positive number')
    ], postCreateCourse)

// Get a specific course by ID
router.route('/:id')
    .get(getCourseById)
    .patch(updateCourseById)
    .delete(deleteCourseById);

module.exports = router;

//utils
--> //.env
MONGO_URL = mongodb+srv://user2000:Ahmed11ss@learn-mongod
b.j8o5kyg.mongodb.net/Code_Zone?appName=learn-MongoD
PORT = 3000

// appError.js
class AppError extends Error {
    constructor(){
        super();
    }
    create(message, statusCode , statusText) {
        this.message = message;
        this.statusCode = statusCode;
        this.statusText = statusText;
        return this;
    }
}

module.exports = new AppError();
// --> httpStatusText.js
const SUCCESS = 'success';
const FAILED = 'failed';
const ERROR = 'error';

```

```

module.exports = {
  SUCCESS,
  FAILED,
  ERROR
};
// session 5 --> index.js
require('dotenv').config({ path: './utils/.env' });
const cors = require('cors');
const httpStatusText = require('./utils/httpStatusText');
const express = require('express');
const app = express();
const { title } = require('node:process');
const dns = require('dns');
dns.setServers(["8.8.8.8", "1.1.1.1"]);
const port = process.env.PORT ;

const mongoose = require('mongoose');

const url = process.env.MONGO_URL ;

mongoose.connect(url).then(() => {
  console.log("Connected to MongoDB");
});

app.use(cors());

app.use(express.json()); // Middleware to parse JSON request bodies

const coursesRouter = require('./routes/courses_routes');
app.use('/api/courses', coursesRouter);

// global middleware for not found router
app.all(/.*/, (req, res) => {

```

```

    res.status(404).json({
      status: httpStatusText.ERROR,
      data: { message: 'Route not found' }
    });
  });

// global middleware for error handling
app.use((error, req, res, next) => {
  res.status(error.statusCode || 500).json({
    status: error.statusText || httpStatusText.ERROR,
    data: { message: error.message },
    code: error.statusCode || 500,
    data: null
  });
});

app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});

```


▼ This is Explanation

1) jSend Specification

الفكرة ببساطة

دي طريقة **Standard** عشان ترجع ال API Response بشكل موحد.

بدل ما كل endpoint يرجع شكل مختلف، بنخلي كله نفس الشكل 

Success

```
{
  "status": "success",
  "data": {
    "user": {
      "name": "Ahmed"
    }
  }
}
```

Error

```
{
  "status": "error",
  "message": "Something went wrong"
}
```

Fail (validation)

```
{
  "status": "fail",
  "data": {
    "email": "Email is required"
  }
}
```

ليه بنستخدمه؟ 💡

- يخلي الـ API predictable
- الفرونت يعرف يتعامل بسهولة
- بيخلي debugging أسهل

عملي في Express 🧪

```
app.get("/user", (req,res) => {res.status(200).json({
  status:"success",
  data: {
    user: { name:"Ahmed" }
  }
});
});
```

🌍 2) dotenv

الفكرة 📌

بنخزن بيانات حساسة (زي DB password) في ملف `env` بدل ما نخطها في الكود

ليه مهم؟ 💡

- أمان 🗝️
- تقدر تغير الإعدادات بدون تعديل الكود
- تستخدم نفس المشروع في dev و production بسهولة

عملي 🧪

1. تثبيت

```
npm install dotenv
```

2. ملف .env

```
PORT=3000  
DB_URL=mongodb://localhost/test
```

3. في الكود

```
require("dotenv").config();console.log(process.env.PORT);
```

3) Pagination

الفكرة

بدل ما ترجع record 1000 مرة واحدة 😊
بنرجعهم على صفحات (page by page)

ليه؟

- ⚡ أداء أفضل
- تقليل الضغط على السيرفر
- أحسن UX

عملي (Mongoose)

```
app.get("/users",async (req,res) => {constpage=req.query.page*1||1;constlimit=req.query.limit*1||5;constskip= (page-1)*limit;constusers=awaitUser.find().skip(skip).limit(limit);res.json({  
  status:"success",  
  results:users.length,  
  data:users
```

```
});  
});
```

4) Mongoose find() options

الفكرة

نقدر نتحكم في البيانات اللي بترجع

أمثلة

select (اختار fields)

```
User.find().select("name email");
```

sort

```
User.find().sort("-createdAt");
```

limit

```
User.find().limit(10);
```

ليه؟

- تقلل حجم البيانات
- تحسن الأداء
- تتحكم في شكل النتائج

5) Error Handling

الفكرة

بدل ما السيرفر يقع  بنمسك الأخطاء ونتعامل معها

عملي

middleware

```
app.use((err, req, res, next) => {res.status(500).json({
  status:"error",
  message:err.message
});
});
```

ليه؟

- يمنع crash
- يحسن UX
- مهم في production

6) 404 Middleware

الفكرة

لو route مش موجود → نرجع 404

عملي

```
app.use((req, res, next) => {res.status(404).json({
  status:"fail",
});
});
```

```
message:"Route not found"
});
});
```

ليه؟ 💡

- يمنع endpoints fake
- مهم للأمان (ما تسيبش السيرفر يرد بأي حاجة)

🌐 7) CORS

الفكرة 📌

السماح للفرونت (React مثلاً) يكلم السيرفر

المشكلة 💡

لو frontend على:

```
localhost:3000
```

و backend:

```
localhost:5000
```

➡ هيترفض الطلب ❌

الحل 🧪

```
npm install cors
```

```
const cors = require("cors"); app.use(cors());
```

ليه مهم؟ 💡

- لازم لأي SPA
- يتحكم مين يقدر يوصل للسيرفر

8) Async Wrapper Middleware

المشكلة 📌

كل شوية تكتب try/catch 😞

الحل 📌

نعمل wrapper يمسك الأخطاء لوحده

عملي 🧪

```
const asyncWrapper = (fn) => { return (req, res, next) => { fn(req, res, next).catch(next); } };
```

استخدامه

```
app.get("/users", asyncWrapper(async (req, res) => { const users = await User.find(); res.json({ status: "success", data: users }); }));
```

ليه؟ 💡

- يقلل الكود
- ينظم المشروع

- best practice 🔥

🧠 سريع Recap

الموضوع	الهدف
jSend	response توحيد شكل
dotenv	حماية البيانات
Pagination	تحسين الأداء
Mongoose options	تحكم في البيانات
Error handling	crash منع
404	routes مش موجودة
CORS	ربط الفرونت بالباك
async wrapper	try/catch تقليل

🚫 1) 404 Error Handler باستخدام

`app.all("*")`

📌 الفكرة

أي request ييجي وميلقاش route مناسب → نرجع 404

🧠 الفرق بين `app.use` و `app.all("*")`

- `app.use` → middleware عام
- `app.all("*")` → **requests** (GET / POST / PUT ...) **يمسك كل**

📝 الشكل الصح


```
app.all("*", (req,res,next) => {const err=newError(`Can't find${req.originalUrl}`);err.  
statusCode=404;next(err);  
});
```

✗ الغلط الشائع

```
app.all("*", () => {})
```

ده syntax غلط + مش بيرجع error أصلاً 😊

💡 ليه بنستخدم الطريقة دي؟

- نخلي 404 يدخل في global error handler
- نحافظ على structure واحد (زي jSend)

🔄 بيحصل ازاي؟ flow

1. request ييجي

2. مش لاقى route

3. يدخل `app.all("*")`

4. يعمل error

5. يروح لل error middleware

⚙️ 2) Better Error Handling (احترافي)

📌 المشكلة

لو كل error بيرجع 500 → ده غلط ✗

📌 الحل



1. Custom Error Class

```
class ApiError extends Error {
  constructor(message, statusCode) {super(message); this.statusCode = statusCode; this.status = statusCode >= 400 && statusCode < 500 ? "fail" : "error";}
}
```

2. استخدام

```
if (!course) {return next(new ApiError("Course not found", 404));}
```

3. Global Error Handler

```
app.use((err, req, res, next) => {res.status(err.statusCode || 500).json({
  status: err.status || "error",
  message: err.message || "Internal Server Error"
});});
```

ليه ده أحسن؟

- بيديك control كامل
- بي فرق بين:
 - fail (user غلط)
 - error (server غلط)
- clean code 

3) Async Functions Error Handling

المشكلة الكبيرة 🐱‍💩

```
app.get("/users", async (req, res) => {const users = await User.find(); // error?
});
```

➡ Express هنا تلقائي error مش بيمسك ❌

الحل التقليدي (وحش) ❌

```
try {
  ...
} catch (err) {next(err);
}
```

الحل الصح (Async Wrapper) 🔥

الكود 🧪

```
const asyncHandler = (fn) => {return (req, res, next) => {fn(req, res, next).catch(next);
};
};
```

الاستخدام 🧪

```
app.get("/users", asyncHandler(async (req, res) => {const users = await User.find(); res.json
({
  status: "success",
  data: users
}));
});
```

بيحصل ايه؟ 🧠

• أي error يحصل جوه async

- `.catch(next)`

• يروح لل error middleware

ليه مهم؟ 💡

• يمنع crash

• يقلل الكود

- في الشركات standard 📁

كلهم مع بعض (Flow كامل) 🔗

```
// 1. Routesapp.get("/courses",asyncHandler(async (req,res) => {constcourses=awaitCourse.find();res.json({ status:"success", data:courses });}));// 2. 404 handlerapp.all("*", (req,res,next) => {next(newApiError("Route not found",404));});// 3. Global error handlerapp.use((err,req,res,next) => {res.status(err.statusCode|500).json({status:err.status||"error",message:err.message});});
```

خلاصة مهمة جداً 🧠

الحاجة	ليه
app.all("*")	مش موجود route يمسك أي
ApiError	تنظيم الأخطاء
asyncHandler	async من errors يمسك
global error middleware	نقطة مركزية لكل الأخطاء

